

Web — the bigger jobs

Six pieces of work, each big enough to take real time and none of them a one-line fix. They are the things that keep costing us afterwards: work that gets undone, work nobody can prove still works, and code that makes every later job slower. Checked in the code today against the latest `dev` (`cb2d905d`).

21 August 2026 · 6 tasks

Roughly in the order they should be done — W1 protects everything after it.

HOW TO READ THIS

What is decided and what is yours

Every task says what is wrong, what the end state must be, and how we will know it is finished. The end state is decided. **How** you get there is yours, and these are deliberately open enough that there is more than one sensible route.

None of these change what a user sees, with one exception clearly marked in W4. Same screens, same behaviour, same look. If any item turns out to need a visible change we have not described, stop on that one and ask rather than deciding.

Each one is a separate pull request. Do not fold them together.

W1 · Make the pipeline able to stop a bad build

W1 Nothing today prevents broken code reaching customers DO FIRST HIGH

The pipeline runs a type check, a linter, a build and the tests. Then it ships — from a step that never looks at whether any of them passed. The linter is explicitly told to carry on regardless, and so are both image builds, so a broken build and a good one look identical on the pull request.

The linter is currently reporting 430 problems, 52 of them errors. Nearly half come from the admin app using a library it never declared — it works only because another app in the monorepo happens to install it. The admin build also ignores the lockfile, so two builds of the same commit can install different software.

DO

Make shipping depend on the checks passing. Nothing is published unless the type check, the linter, the build and the tests all succeeded.

Clear the 430 lint problems and declare the missing dependency in the admin app properly, then turn the linter from a warning into a gate. Clearing them first and gating second — the other order blocks everyone.

Remove the “carry on after a failure” setting from both image builds so a failed build fails the pull request.

Make the admin build honour the lockfile, like the client panel does.

FINISHED WHEN

A pull request that breaks the type check, fails the linter, breaks the build, fails a test, or breaks an image build cannot be merged and cannot be deployed — and you can show us each of those five cases going red.

WATCH OUT

This has been the top item on three separate lists since 10 July and has not moved. Until it is done, anything fixed on the web side can silently come back — two paid fixes were merged in May, undone by one merge in June, and nobody noticed for eight weeks.

The backend already has this. Copy what it does rather than designing something new.

W2 · Put a real test suite on the scoring code

W2 Both web apps have two test files and both assert that true is true HIGH

That is the entire test suite. The numbers that tell a customer whether their worker is at risk — RULA, REBA and RAMP scoring, the zone each score falls into, and the colour that goes with it — are not covered by a single test. This is also why W1 matters and why the two belong together: a gate that runs no tests gates nothing.

DO

Start with the scoring, not the screens. RULA and REBA score calculation, RAMP score calculation, the score-to-zone mapping and the score-to-colour mapping. Table-driven tests: a list of inputs and expected outputs, so adding a case later is one line.

Cover the boundaries specifically — the exact value where a score changes zone, and the values on either side of it. That is where the two colour rules disagree today, so the tests will disagree too; write them against the results page and note the difference rather than picking a winner.

Then the shared formatting helpers: durations, percentages, and the time formatting used on the results tables. Cheap to test and they appear on every screen.

Then one rendering test per results screen that proves the page draws with real data and with data that has a missing limb in it.

FINISHED WHEN

The scoring functions have full branch coverage, the suite runs in the pipeline, and it goes red when you deliberately change a threshold by one.

WATCH OUT

Do not chase a coverage percentage across the whole app. Cover the numbers a customer relies on, properly. Everything else can come later.

The backend went from no tests to a full suite in about a week this month. Ask them what they did before you start — the setup work is already solved.

Open Work reference: G3

W3 · Delete the v1 screens

W3 The old interface is still in the codebase and it is now the main source of confusion **MULTI-DAY**

v2 was switched on everywhere in July. The v1 pages are still there, behind switches that can no longer be turned on, and a handful of old pages are still reachable by typing the address.

This is not tidiness. Most of the wrong findings in our last three audits came from someone reading a dead screen and reporting a bug that no customer can reach. It costs us time every single review, and it will cost you time every time you search the codebase.

DO

Confirm with us first that v2 is not being rolled back. Then remove the v1 pages, the switches that used to select them, and the addresses that still reach them.

While you are in there, three specific things go with it: the three places that switch off type checking on data coming back from the backend — one of those caused the “Measured this month: 0” bug — and the unused file that defines the correct spelling of the cached assessment keys while the whole app uses a misspelled one. Delete the unused file or adopt it everywhere; do not leave both.

Also remove the dead admin interface: the commented-out bulk archive action, the removed notifications bell whose button would not have worked, and the “?” icon that looks pressable and only has a tooltip.

FINISHED WHEN

Nothing in the codebase renders a v1 screen, no old address resolves, no type checking is switched off on backend data, and the misspelled cache keys exist in exactly one form.

WATCH OUT

Do this after W1. Without a working gate, a removal this size is the exact kind of change that breaks something quietly.

Every address you retire should be listed in the pull request, so we can check none of them are in a customer’s bookmarks.

Open Work reference: H2, H3, H4, H6

W4 · One way to show an error

W4 The app knows what went wrong and shows the user a developer's note HIGH MULTI-DAY

When a page fails to load, the customer sees text a developer wrote for themselves. One screen literally reads "React query error thrown manually from ArchivedSessionsTable". There are twenty-seven of these in the code; about eight are on screens a customer can still reach.

When the backend rejects a form and names the field, the client panel reads only the general message and shows a vague — sometimes empty — notification, with nothing marking the field.

And the catch-all error screen makes it worse. It calls its own reset while it is still drawing, which causes the error again, which resets again, until the tab freezes. Its "Try again" button only redraws the page — it never asks the server a second time — so the same error comes straight back. Both apps. Its text is hardcoded English in every language.

DO

Build one way to turn a failed request into something on screen, and use it everywhere. One place that reads the server's reason, one place that decides what the user sees.

Where the backend names a field, the message goes on that field. The duplicate-email case already does this — copy it. Where it does not name a field, show the server's reason, not a placeholder.

Fix the reset loop: the error screen must not reset itself while rendering.

Make "Try again" actually re-request the data. Translate that screen in both apps, in all five languages.

Then replace all twenty-seven developer messages with the shared handling. Delete the ones on screens that no longer render rather than converting them — but only after W3, or you will convert dead code.

FINISHED WHEN

No screen in either app can show a developer's message. A rejected form marks the field. The error screen cannot loop, its Try again re-fetches, and it is translated.

WATCH OUT

This is the one task with a visible change, and it is intentional: users will start seeing real reasons instead of placeholders. Nothing else about the screens changes.

The backend has matching work to do — several endpoints still return a duplicate value, a missing field and a genuine crash in exactly the same shape. Agree the shape with the backend developer before you build against it, or you will build twice. The work-cycle import was fixed this month and is the example to follow.

Open Work reference: E1, E3, E4, E11

W5 · Make the build reproducible

W5 Nobody knows how production gets the backend's address HIGH

Neither app is given the backend's address properly when it is built. An image built the normal way has no valid address in it. And yet production works — which means something undocumented is supplying it and nobody knows what.

This is the difference between “we deploy” and “we can deploy again if the machine dies.”

DO

Find out where production actually gets it from today. Write that down before changing anything — that answer alone is worth the task.

Then wire it in properly, so the address is a declared input to the build for both apps and both modes.

Then make the build fail, loudly and early, if it is missing — rather than producing an image that looks fine and points nowhere.

FINISHED WHEN

You can build both apps from a clean checkout, on a machine that has never built them before, and get images that work; and an image built without the address fails the build instead of shipping.

Open Work reference: [G4](#)

W6 · Small, but ship it with the rest

W6 A developer debugging panel is shipped to customers, and console errors bury the real ones **FIX**

The React Query devtools panel is rendered in production in both apps, and it is installed as a normal dependency rather than a development one, which is why nobody noticed.

Separately, both apps log errors to the browser console on every page load. Harmless in themselves, but they bury the ones that matter, and every one of them is something we would want to know about.

DO

Move the devtools to a development dependency and render it only in development.

Then clear the console on first load of every main screen in both apps. Each one is either a real problem — fix it — or noise, in which case remove what produces it. Do not silence them.

Send us a list of what each one turned out to be. Some of these are likely to be findings in their own right.

FINISHED WHEN

A production build renders no devtools panel, and a first load of Home, Measurements, Assessments and Team produces a clean console in both apps.

Open Work reference: H1, H12

WHEN YOU ARE DONE

Send back

Per task: done or not, and what you found that we did not describe. W5's answer and W6's console list are deliverables in themselves — do not skip them because the code change was small.

W1 first, and tell us the day it is merged. Everything else is worth more once it exists.